Morphologie mathématique et étiquetage

Fiche de travaux pratiques • MATLAB / Python

Objectifs

Nettoyer une image binaire, comparer les opérateurs morphologiques, identifier les composantes connexes et compter les objets.

Image utilisée dans les trois exercices : cellbin.bmp, déjà binaire, avec des cellules blanches sur fond noir. La placer dans le dossier de travail et la renommer cellbin.bmp si nécessaire. Réaliser le TP sous MATLAB ou Python.

Convention : les objets valent 1 et le fond 0. Pour les opérations morphologiques, utiliser un élément structurant centré de taille impaire et un fond nul à l'extérieur de l'image.

Exercice 1 — Érosion et dilatation

1. Lire cellbin.bmp dans la variable BW et l'afficher. Conserver cette image initiale pour les comparaisons. Définir un élément structurant carré de taille 3 × 3.

2. Appliquer séparément une érosion et une dilatation à BW. Afficher BW et les deux résultats côte à côte.

3. Décrire les effets sur la taille des régions, les petits objets et les espaces entre objets.

4. Recommencer avec des carrés 5 × 5 et 9 × 9, toujours à partir de BW. Quels objets disparaissent ou se rejoignent ?

5. Remplacer le carré 3 × 3 par une croix 3 × 3. Comparer les résultats et expliquer le rôle de la forme de l'élément structurant.

Exercice 2 — Ouverture et fermeture

1. Appliquer séparément une ouverture et une fermeture à BW avec un carré 3 × 3. Afficher les résultats.

2. Retrouver l'ouverture en enchaînant une érosion puis une dilatation. Retrouver la fermeture en inversant cet ordre. Utiliser le même élément et les mêmes conditions aux bords.

3. Vérifier que ces constructions donnent les mêmes résultats que les fonctions dédiées. Quels défauts chaque opération peut-elle corriger ?

4. Appliquer une ouverture 3 × 3 puis une fermeture 3 × 3. Nommer le résultat BW_net. Observer les fragments, les trous et les contacts. Tester ensuite une fermeture 9 × 9 : quelles régions fusionnent ?

Exercice 3 — Étiquetage et comptage des composantes

1. Reprendre BW et BW_net (ouverture 3 × 3 puis fermeture 3 × 3). Les étiqueter en 8-connexité. Afficher les régions en couleurs et les nombres de composantes, sans compter le fond.

2. Comparer les deux résultats. Repérer les régions supprimées, séparées ou fusionnées par la morphologie. Le nettoyage améliore-t-il le comptage des cellules ?

3. Calculer l'aire de chaque composante de BW_net, en pixels. Afficher ces aires et repérer la plus grande région.

4. Choisir un seuil d'aire A_min pour éliminer les petites régions parasites. Justifier ce choix, supprimer les composantes d'aire inférieure à A_min, puis réétiqueter et compter.

5. Étiqueter ce même masque filtré en 4-connexité. Comparer au résultat en 8-connexité et expliquer les éventuelles différences.

6. Le nombre de composantes correspond-il nécessairement au nombre réel de cellules ? Expliquer le cas des cellules qui se touchent et celui des petits fragments isolés.

Tableau des fonctions utiles

BW : image binaire initiale ; BW_net : masque après morphologie ; SE : élément structurant ; L : étiquettes ; n : nombre de composantes hors fond.

Opération	MATLAB	Python
Lire l'image binaire	BW = imread("cellbin.bmp")	BW = cv2.imread("cellbin.bmp", 0) > 0
Dimensions / valeurs	size(BW) ; unique(BW)	BW.shape ; np.unique(BW)
Afficher / comparer	subplot(1,m,k) ; imshow(BW) title("Titre")	plt.subplot(1,m,k) plt.imshow(BW, cmap="gray")
Carré k × k	SE = ones(k,k)	SE = np.ones((k,k), dtype=bool)
Croix 3 × 3	SE = [0 1 0; 1 1 1; 0 1 0]	SE = ndi.generate_binary_structure(2,1)
Érosion*	imerode(BW,SE)	ndi.binary_erosion(BW, structure=SE)
Dilatation*	imdilate(BW,SE)	ndi.binary_dilation(BW, structure=SE)
Ouverture*	imopen(BW,SE)	ndi.binary_opening(BW, structure=SE)
Fermeture*	imclose(BW,SE)	ndi.binary_closing(BW, structure=SE)
Étiqueter	[L,n] = bwlabel(BW,4) [L,n] = bwlabel(BW,8)	S = ndi.generate_binary_structure(2,c) L,n = ndi.label(BW, structure=S) c = 1 : 4-connexité ; c = 2 : 8-connexité
Colorier les régions	imshow(label2rgb(L))	plt.imshow(color.label2rgb(L, bg_label=0))
Mesurer les aires	stats = regionprops(L,'Area')	aires = np.bincount(L.ravel())[1:]
Garder les aires ≥ A_min	ids = find([stats.Area] >= A_min); BW_filtre = ismember(L,ids);	ids = np.flatnonzero(aires >= A_min) + 1 BW_filtre = np.isin(L, ids)

Modules nécessaires

Python : numpy (np), matplotlib.pyplot (plt), cv2, scipy.ndimage (ndi) et skimage.color (color). Sous MATLAB : Image Processing Toolbox.

* Pour comparer exactement les opérations aux bords : ajouter une bordure de zéros assez large au masque avant traitement, puis recadrer au format initial (padarray sous MATLAB, np.pad sous Python). Pour deux opérations successives, prendre une marge au moins égale à k − 1 pour un carré k × k.